



Abschlussprüfung Winter 2024/25

Fachinformatiker für Anwendungsentwicklung
Dokumentation zur betrieblichen Projektarbeit

WiseScan

App für Erfassung von Wartungsarbeiten

Abgabetermin: Lübeck, den 11.12.2024

Prüfungsbewerber:

Svenja Katharina Czayka

Prüflingsnummer: 12003

Schwartauer Allee 74a

23554 Lübeck



Ausbildungsbetrieb:

ananas.codes

Lindenstraße 8

23558 Lübeck

Inhaltsverzeichnis

1.	Einleitung	1
1.1.	Projektbeschreibung	1
1.2.	Projektziel	1
1.3.	Projektbegründung	1
1.4.	Projektschnittstellen	2
1.5.	Projektabgrenzungen	2
1.6.	Prozessabläufe	3
2.	Projektplanung	3
2.1.	Zeitplanung und Projektphasen	4
2.2.	Ressourcenplanung	4
2.3.	Analysen	4
2.3.1.	Ist-Analyse	4
2.3.2.	Soll-Konzept	5
2.3.3.	Wirtschaftlichkeit Analyse	6
2.4.	Workflow	6
2.5.	Kosten für das Projekt	7
3.	Entwurf und Design	7
3.1.	Systemarchitektur	7
3.2.	Datenbankdesign	8
3.3.	API-Design und Aufbau	9
3.3.1.	API-Endpunkte	10
3.3.2.	Sicherheitsaspekte	10



3.4.	Benutzeroberfläche (UI) und Benutzererfahrung (UX) der App	11
3.4.1.	Entscheidung für das Expo Framework	11
3.5.	Designrichtlinien	12
3.5.1.	Farbschema und Typografie	12
3.5.2.	Responsive Design	13
4.	Durchführung und Implementierung	14
4.1.	Zielplattform und Technologie Stack	14
4.2.	Durchführung und Implementierung der Datenbank	14
4.3.	Durchführung und Implementierung der API	14
4.4.	Durchführung und Implementierung der App	15
4.5.	Testphase	17
5.	Ergebnisse und Fazit	17
5.1.	Fazit	18
5.2.	Soll-Ist-Vergleich	17
5.3.	Lessons Learned	18
5.4.	Ausblick	18
a.	Anlagen	I
a.1.	Prozessabläufe	I
a.2.	Prozesse für Handwerker	I
a.3.	Zusatzprozesse für Administratoren	II
a.4.	Use-Case	III
a.5.	Grobe Projektphase	IV



a.6.	Detaillierte Projektphase	IV
a.7.	Ressourcenplan	VIII
1)	Menschliche Ressourcen	VIII
2)	Hardware	VIII
3)	Software	VIII
4)	Technologien und Programmiersprachen	IX
5)	Sonstige Ressourcen	IX
6)	Zeitplan	IX
a.8.	Break-Event Point	X
a.9.	Kosten des Projekts	X
a.10.	Datenfluss	XII
a.11.	EER-Model	XIII
a.12.	API-Aufbau	XIV
a.13.	Wireframe	XVI
a.14.	Mockup	XVII
a.15.	Farbschema	XVII
a.16.	Ablaufdiagramm app	XVIII
a.17.	Auszüge aus dem Code	XIX
a.18.	Fehlercodeliste der API	Fehler! Textmarke nicht definiert.
a.19.	Ordnerstruktur der App	XXV

Abbildung Verzeichnis

Abbildung 1 Use-Case	III
Abbildung 2 Sequenzdiagramm Datenfluss	XII



Abbildung 3 EER-Model.....	XIII
Abbildung 4 API-Ordnerstruktur.....	XIV
Abbildung 5 Farbschema in der App.....	XVII
Abbildung 6 Ablaufdiagramm der App	XVIII
Abbildung 7 API-Index.....	XIX
Abbildung 8 API ReactController.....	XX
Abbildung 9 API-Gateway.....	XX
Abbildung 10 API ErrorHandler	XXI
Abbildung 11 Auszug API Service in der App	XXI
Abbildung 12 ConfigScreen in der App	XXII
Abbildung 13 Hauptlayout App	XXII
Abbildung 15 Beispiel für usestate aus der screen protollscreen	XXIII
Abbildung 14 Richtlinien für Responsiven Text und Farbetext	XXIV



Abkürzungsverzeichnis

API	Application Programming Interface
PDO	PHP Data Objects
App	Application (Anwendung)
UI	User Interface (Benutzeroberfläche)
UX	User Experience (Benutzererfahrung)
SQL	Structured Query Language
QR	Quick Response (QR-Code)
SSL	Secure Sockets Layer
XSS	Cross-Site Scripting
PHP	Hypertext Preprocessor



1. Einleitung

Diese Projektdokumentation beschreibt das Abschlussprojekt, das im Rahmen der Umschulung zum Fachinformatiker für Anwendungsentwicklung durchgeführt wurde. Das Projekt entstand während eines Praktikums bei ananas.codes, einem Unternehmen, das Lösungen zur Zeiterfassung und Dokumentation anbietet. Ziel war es, eine App zu entwickeln, die den Anforderungen von ananas.codes sowie einer externen Facility-Management-Firma entspricht. Diese Dokumentation umfasst alle Phasen des Projekts, von der Planung und Analyse über die Umsetzung bis hin zur Bewertung der Ergebnisse.

1.1. Projektbeschreibung

Für eine Facility-Management-Firma, die Wartungsarbeiten bei verschiedenen Kunden koordiniert, soll eine digitale Lösung entwickelt werden, die den Wartungsprozess optimiert und vereinfacht. Der Autor ist für die vollständige Planung, Entwicklung und Implementierung der App verantwortlich, welche es Handwerksfirmen ermöglicht, Wartungsarbeiten digital zu dokumentieren und die Daten in eine zentrale Datenbank zu übertragen. Um die Lösung auch für ähnliche Projekte nutzbar zu machen, wird die App als lizenzbasierte Lösung angeboten. Die Umsetzung erfolgt hauptsächlich bei ananas.codes, wo enge Zusammenarbeit und schnelle Entscheidungsprozesse gewährleistet sind.

1.2. Projektziel

Ziel ist die Entwicklung einer App, mit der Handwerksfirmen Wartungsarbeiten effizient und digital per QR-Code erfassen können. Im Rahmen des Projekts entwickelte App WiseScan modernisiert den bisherigen papierbasierten Prozess und ermöglicht eine zentrale Speicherung der Wartungsdaten, wodurch die Effizienz und Genauigkeit gesteigert wird.

1.3. Projektbegründung

Im Gegensatz zur papierbasierten Wartungsdokumentation, die in der Verwaltung aufwendiger und weniger flexibel ist, bietet eine digitale Lösung deutliche Vorteile. Sie ermöglicht eine präzisere Nachverfolgbarkeit, senkt langfristig Kosten und unterstützt



eine zuverlässigere Kontrolle, wodurch potenzielle Betriebsunterbrechungen reduziert werden können.

1.4. Projektschnittstellen (Stakeholder wissen geholt)

Das Projekt umfasst die folgenden Schnittstellen:

- **Kunde (Facility Management Firma):** Der Kunde ist verantwortlich für die Koordination der Wartungsarbeiten und definiert die Anforderungen an die App.
- **ananas.codes Team:** Die enge Zusammenarbeit im kleinen Team ermöglicht eine effektive Kommunikation und schnelle Entscheidungswege für eine effiziente Projektumsetzung.
- **Handwerkerfirmen (App-Nutzer):** Die App muss an die Bedürfnisse der Handwerkerfirmen angepasst sein.
- **Zentrale Datenbank:** Über eine sichere Schnittstelle werden die Wartungsdaten an eine zentrale Datenbank übermittelt
- **Technische Infrastruktur und Netzwerksicherheit:** Eine sichere technische Infrastruktur gewährleistet den zuverlässigen Betrieb der App und die Einhaltung der Datenschutzanforderungen.
- **Lizenzmanagement:** Eine Schnittstelle zur Verwaltung und Überwachung der Lizenzen erlaubt die Nutzung der App bei weiteren Projekten und Unternehmen.

1.5. Projektabgrenzungen

Im Rahmen des Projekts wurden bestimmte Aspekte bewusst ausgeschlossen, um den Fokus auf die zentrale Softwareentwicklung und die Integration der Anwendung zu legen.

Hardware und Sensortechnik	Die Bereitstellung und Entwicklung von Hard-ware-Komponenten wie Barcode-Generatoren oder mobilen Geräten waren nicht Bestandteil des Projekts. Der Fokus lag auf der Softwareentwicklung (App, API, Datenbank) und deren Integration mit bestehender Hardware.
----------------------------	---



Schulungen	Die Schulung der Endnutzer lag außerhalb des Projektumfangs. Das Projekt beschränkte sich auf die technische Bereitstellung und Dokumentation.
Backend-Infrastruktur	Die Verantwortung des Projekts endete bei der Entwicklung und Anbindung der App-Datenbank. Die Wartung externer Systeme (wie Server oder zusätzliche Datenbanken) oblag dem Auftraggeber oder einem separaten IT-Team.
Markteinführung und Lizenzierung	Der Projektumfang umfasste nicht die Marketing- und Lizenzierungsprozesse für die Markteinführung, da der Schwerpunkt auf der technischen Entwicklung und Implementierung lag.
Nachträgliche Erweiterungen:	Anpassungen oder Erweiterungen nach Projektabschluss wurden nicht berücksichtigt und wären Gegenstand zukünftiger Projekte.
Lifecycle	
security	
Anforderung	

1.6. Prozessabläufe

In diesem Abschnitt werden die spezifischen Prozesse der Anwendung detailliert beschrieben, die auf die beiden Hauptakteure – Handwerker und Administrator – zugeschnitten sind. Eine ausführliche Beschreibung der Prozesse ist im Anhang zu finden unter *Prozessabläufe auf Seite*. Zusätzlich wurde zur besseren Visualisierung der Abläufe ein *Use-Case* erstellt, welches ebenfalls im Anhang hinterlegt ist.

2. Projektplanung

Hier wird die Planung des Projekts detailliert beschrieben, einschließlich der Analysen, der notwendigen Ressourcen und der Zeitrahmen, die zur erfolgreichen Umsetzung der digitalen Wartungsdokumentation erforderlich sind.



2.1. Zeitplanung und Projektphasen

Für das Projekt wurden verschiedene Phasen mit Unterpunkten definiert. Insgesamt waren 80 Stunden angesetzt, wovon 4 Stunden als Puffer für mögliche Hindernisse eingeplant wurden. Eine grobe Zeitplanung wurde im Vorbeginn des Projektes (im Anhang unter **Fehler! Verweisquelle konnte nicht gefunden werden.** erstellt, aus dieser groben Zeitplan wurde ein detaillierte Projektphasenplan erstellt dieser ist im Anhang *unter Fehler! Verweisquelle konnte nicht gefunden werden.* hinterlegt.

2.2. Ressourcenplanung

Die Planung umfasst die benötigten Hard- und Software-Ressourcen sowie das eingesetzte Personal. Um Kosten zu senken, wurde darauf geachtet, lizenzfreie Software zu verwenden und vorhandenes Fachwissen optimal einzusetzen. Eine detaillierte Auflistung ist in den Anlagen (*Fehler! Verweisquelle konnte nicht gefunden werden.Fehler! Verweisquelle konnte nicht gefunden werden.Ressourcenplanung*) hinterlegt

2.3. Analysen

Die Analysen umfassen die **Ist-Analyse**, das **Soll-Konzept** sowie die **Wirtschaftlichkeitsanalyse**, um den aktuellen Stand zu bewerten, die Zielvorgaben zu definieren und die Kosten-Nutzen-Verhältnisse des Projekts zu prüfen.

2.3.1. Ist-Analyse

Aktueller Prozess

Die Dokumentation der Wartungsarbeiten erfolgt manuell in Papierform.

Probleme des aktuellen Systems

➤ Fehleranfälligkeit:

- Handschriftliche Einträge können unleserlich oder unvollständig sein.
- Übertragungsfehler beim Übertragen von Papier in digitale Systeme.

➤ Zeitaufwand:

- Manuelle Erfassung ist zeitintensiv.
- Nachträgliche Digitalisierung der Papierdokumente erfordert zusätzlichen Aufwand.



➤ Ineffizienz:

- Schwierige Nachverfolgung und Auswertung der Wartungshistorie.
- Verzögerungen bei der Informationsweitergabe an andere Abteilungen.

➤ Ressourcenverbrauch:

- Hoher Papierverbrauch.
- Physischer Speicherplatz für Archivierung notwendig.

➤ Mangelnde Flexibilität:

- Schwierige Anpassung an sich ändernde Anforderungen.
- Eingeschränkte Möglichkeiten für schnelle Datenanalysen.

2.3.2. Soll-Konzept

Die oben angesprochenen Probleme sollen behoben werden. Dazu bietet sich eine digitale Lösung an. Dabei sich folgende Anforderungen und Funktionen herauskristallisiert:

➤ **Zentrale Datenbank:**

- Speicherung aller Wartungsdaten in einer gesicherten, zentralen Datenbank für Übersichtlichkeit und Nachverfolgbarkeit.

➤ **QR-Code-Dokumentation:**

- Verwendung von QR-Codes zur schnellen Identifikation und Dokumentation von Wartungen durch die Handwerker.

➤ **Echtzeitsynchronisierung:**

- Automatische Synchronisierung der Daten mit der zentralen Datenbank für aktuelle Wartungsinformationen.

➤ **Benutzerfreundliche Oberfläche:**

- Intuitive Bedienung, die den spezifischen Bedürfnissen der Handwerker gerecht wird.

➤ **Rollen- und Rechteverwaltung:**

- Differenzierte Zugriffsrechte für Handwerker und Facility-Manager zur Sicherstellung von Datenschutz und Datensicherheit.

➤ **Exportfunktionen:**

- Automatische Dokumentation und einfache Datenexporte für Berichterstattungen.

➤ **Lizenzbasierte Nutzung:**



- Flexibles Lizenzmodell zur Nutzung der App in weiteren Projekten oder Unternehmen.

2.3.3. Wirtschaftlichkeit Analyse

In diesem Abschnitt wird die Wirtschaftlichkeit des Projekts analysiert. Dazu werden die Entwicklungskosten, laufenden Kosten sowie die potenziellen Einnahmen betrachtet. Ein besonderer Fokus liegt auf der Berechnung des Break-Even-Points (BEP), der zeigt, wie viele Monatslizenzen erforderlich sind, um die Investitionskosten zu decken.

Kostenanalyse

➤ **Entwicklungskosten:**

- Stundensatz: 120,00€
- Projektzeit: 80 Stunden
- Gesamtkosten Entwicklung: 9.600,00 €

➤ **Laufende Kosten:**

- Betreuung: In den 49,00 € pro Kundenlocation enthalten

➤ **Einnahmenanalyse**

- Lizenzgebühr: 49,00 € pro Kundenlocation pro Monat
- Laufzeit: 2 Jahre

Break-Even-Point

Um die Entwicklungskosten zu decken wurde folgende Rechnung gemacht

$$9.600,00 \text{ €} / (49,00 \text{ € pro Monat}) \approx 196 \text{ Monats – Lizenzen}$$

Potenzielle Einnahmen

➤ **Mindesteinnahmen:**

$$19 \text{ Kunden} * 49,00 \text{ €} * 24 \text{ Monate} = 22.344,00 \text{ €}$$

- **Tatsächliche Einnahmen:** könnten höher sein, abhängig von der genauen Anzahl der Locations pro Kunde.

Im Anhang ist unter *Break-Event Point* eine Grafik hinterlegt.

2.4. Workflow

Dieser Workflow beschreibt die zentralen Schritte im Entwicklungsprozess der App:

➤ **Anforderungsanalyse und Planung**

- Anforderungen mit dem Kunden definieren und Lastenheft erstellen.



- Projektplanung und Zeitleiste festlegen.
- **Konzeption und Design**
 - Systemarchitektur und Datenmodell entwickeln.
 - Benutzeroberfläche und UX-Design entwerfen.
- **Entwicklung der App und API**
 - Front-End- und Back-End-Entwicklung durchführen.
 - Anbindung an die zentrale Datenbank und Implementierung von Sicherheitsmechanismen.
- **Tests und Qualitätssicherung**
 - Benutzerakzeptanztests durchführen.
 - Fehlerbehebung und Optimierung basierend auf Benutzerfeedback.
- **Bereitstellung**
 - App auf Android-Geräten.

2.5. Kosten für das Projekt

Eine Aufstellung der Kosten finden Sie im Anhang unter *Kosten des Projekts*

3. Entwurf und Design

Im Rahmen der Systementwicklung wurden sorgfältig durchdachte Entwurfsmethoden und Designprinzipien angewandt, um eine optimale Funktionalität und Benutzerfreundlichkeit zu gewährleisten. Der Entwurfsprozess umfasste mehrere Schlüsselschritte, die von der Systemarchitektur über das Datenbankdesign bis hin zur Benutzeroberfläche und API-Entwicklung reichen. Diese strukturierten Ansätze trugen dazu bei, ein stabiles und skalierbares System zu schaffen, das sowohl den aktuellen Anforderungen als auch zukünftigen Erweiterungen gerecht wird.

3.1. Systemarchitektur

Der Datenfluss des Systems ist klar definiert, um eine effiziente und sichere Verarbeitung von Informationen zu gewährleisten:

- **Kommunikation:** Die App interagiert über eine API mit einer zentralen MySQL-Datenbank.



- **Datenerfassung:** Handwerker erfassen Wartungsarbeiten entweder durch Scannen eines QR-Codes oder durch manuelle Eingabe.
- **Datenvalidierung und Übermittlung:** Die App validiert die Eingaben und überträgt sie anschließend an die zentrale Datenbank.
- **Administration:** Administratoren haben die Möglichkeit, die App-Konfiguration sowie Benutzerzugänge und API-Kennwörter zu verwalten.

Ein **Sequenzdiagramm**, das den Datenfluss und die Interaktion der Systemkomponenten veranschaulicht, ist zur besseren Verständlichkeit im Anhang hinterlegt unter.

3.2. Datenbankdesign

Um eine effiziente und skalierbare Lösung zu gewährleisten, wurde die Datenbankstruktur sorgfältig entworfen. Ziel war es, Redundanzen zu minimieren, eine klare Trennung der Daten zu ermöglichen und zukünftige Erweiterungen problemlos umsetzen zu können. Die Struktur orientiert sich an den Prinzipien der Normalisierung, wobei die Funktionalität und Performance im Mittelpunkt stehen. Die Beziehung zwischen den verschiedenen Entitäten wurde so gestaltet, dass alle Geschäftsprozesse wie die Verwaltung von Kunden, Standorten, Wartungen und Objekten abgebildet werden können. Ein EER-Diagramm befindet sich unter *EER-Model* im Anhang.

Tabellenstruktur und Beziehungen

Kunden(client)	
Attribute:	Kunden-ID (Primärschlüssel), Name, Adresse, Kontaktinformationen.
Beziehung:	Ein Kunde kann mehrere Standorte haben, daher besteht eine 1: n-Beziehung zu den Standorten.
Unternehmen (company)	
Attribute	Unternehmens-ID (Primärschlüssel), Name, Kontaktinformationen.
Beziehung	Ein Unternehmen kann mehrere Mitarbeiter beschäftigen, daher besteht eine 1: n-Beziehung zu den Mitarbeitern
Standorte (location)	
Attribute	Standort-ID (Primärschlüssel), Kunden-ID (Fremdschlüssel), Adresse



Beziehung	Jeder Standort gehört zu einem Kunden (n:1) und kann mehrere Wartungen haben (1: n).
Wartungen (maintenance)	
Attribute	Wartungs-ID (Primärschlüssel), Standort-ID (Fremdschlüssel), Mitarbeiter-ID (Fremdschlüssel), Datum/Uhrzeit des Scannens
Beziehung	Jede Wartung gehört zu einem Standort und wird von einem Mitarbeiter durchgeführt (n:1)
Objekte (object_list_name)	
Attribute	Objekt-ID (Primärschlüssel), Bezeichnung, Typ
Beziehung	Ein Objekt kann an mehreren Standorten gewartet werden, daher gibt es eine 1: n-Beziehung zur Objektliste
Objektlisten (object_list)	
Attribute	Objektlisten-ID (Primärschlüssel), Objekt-ID (Fremdschlüssel), Standort-ID (Fremdschlüssel), QR-Code
Beziehung	Diese Tabelle verknüpft Objekte mit Standorten und hat somit eine n:m-Beziehung zwischen Objekten und Standorten
Protokolle (protocol)	
Attribute	Protokoll-ID (Primärschlüssel), Wartungs-ID (Fremdschlüssel), Datum/Uhrzeit der Wartung.
Beziehung	Jedes Protokoll gehört zu einer Wartung (1: n).
Mitarbeiter (worker)	
Attribute	Mitarbeiter-ID (Primärschlüssel), Name, Identifikationsnummer, Unternehmens-ID (Fremdschlüssel).
Beziehung	Ein Mitarbeiter gehört zu einem Unternehmen und kann mehrere Wartungen durchführen (1: n). Hier er-model einfügen

Normalform (3NF):

- Jede Tabelle enthält ausschließlich Daten, die direkt miteinander in Beziehung stehen.
- Fremdschlüssel sichern die Integrität und verknüpfen die Daten eindeutig.
- Redundanzen wurden durch die Aufteilung in separate Tabellen minimiert.



3.3. API-Design und Aufbau

Die API wurde entwickelt, um eine reibungslose Kommunikation zwischen der mobilen App und der zentralen Datenbank zu gewährleisten. Sie bietet Endpunkte für CRUD-Operationen (Create, Read, Update, Delete) und ermöglicht eine effiziente Verarbeitung der Wartungsdaten sowie der zugehörigen Informationen.

3.3.1.API-Endpunkte

➤ **Maintenance (Wartungen)**

- **POST /maintenance:**
erstellt einen neuen Wartungseintrag, sofern für den angegebenen Tag noch keiner existiert.
- **GET /maintenance/{id}:**
ruft die Details eines bestimmten Wartungseintrags anhand der Wartungs-ID ab.
- **DELETE /maintenance/{id}:**
löscht einen Wartungseintrag aus der Datenbank.

➤ **Worker (Mitarbeiter)**

- **GET /worker/{ workerIdentifier}:**
ruft die Informationen eines Mitarbeiters anhand seiner Mitarbeiterkennung ab.

➤ **ObjectList (Objektlisten)**

- **GET /objectlist/{ objectIdentifier}:**
ruft die Informationen eines Objekts anhand seines Kennung ab.

3.3.2.Sicherheitsaspekte

➤ **Authentifizierung**

- **API-Schlüssel:** Jede Anfrage an die API erfordert einen gültigen API-Schlüssel, der im Header der Anfrage übermittelt wird. Ohne diesen Schlüssel wird der Zugriff verweigert.

➤ **Sicherer Datenverkehr**

- **SSL/TLS-Verschlüsselung:** Alle Daten, die zwischen der App und der API ausgetauscht werden, sind durch SSL/TLS verschlüsselt.

➤ **Datenbankkommunikation**



- **PDO (PHP Data Objects):**

Für die Kommunikation mit der Datenbank wurde PDO verwendet. PDO bietet Vorteile wie:

- Automatische Unterstützung für Prepared Statements.
- Portabilität: Die API kann bei Bedarf leicht auf andere Datenbankmanagementsysteme umgestellt werden.
- Fehlerbehandlung: PDO unterstützt Ausnahmen (Exceptions), die eine detaillierte Fehleranalyse ermöglichen.

➤ **Validierung und Schutz vor Sicherheitslücken**

- **Eingabedaten-Validierung:** Jede Anfrage wird geprüft, um sicherzustellen, dass nur korrekte und vollständige Daten verarbeitet werden.

➤ **Logging und Monitoring**

- Fehlerhafte Anfragen sowie verdächtige Zugriffsversuche werden protokolliert, um Sicherheitsvorfälle frühzeitig erkennen zu können.

Diese Maßnahmen stellen sicher, dass die API sowohl leistungsfähig als auch sicher ist, und schützen die sensiblen Daten vor unbefugtem Zugriff und Manipulation.

3.4. Benutzeroberfläche (UI) und Benutzererfahrung (UX) der App

Um die Visualisierung der Benutzeroberfläche zu unterstützen, wurden ein Wireframes und darauf basierend ein Mockups erstellt. Das Wireframes bieten eine vereinfachte Darstellung der Layoutstruktur und Navigation, während die Mockups detaillierte Einblicke in die Gestaltung und das endgültige Design liefern. Beide sind im Anhang hinterlegt und dienen als Leitfaden für die Implementierung der Benutzeroberfläche. Zusätzlich wurde ein *Ablaufdiagramm der App* erstellt und befindet sich ebenfalls im Anhang.

3.4.1. Entscheidung für das Expo Framework gegen ionic vergleichen

Für die Entwicklung der mobilen App wurde das **Expo Framework** in Kombination mit **React Native** gewählt. Diese Entscheidung wurde getroffen, um eine effiziente und plattformübergreifende Entwicklung zu ermöglichen. Expo bietet eine Vielzahl an **vorgefertigten Komponenten und APIs**, die die Entwicklungszeit verkürzen und die Wartung vereinfachen.



Vorteile von Expo:

- **Schnelle Entwicklung:** Expo bietet eine schnelle Entwicklungsumgebung, da viele wichtige Funktionen (wie Push-Benachrichtigungen, GPS-Integration) bereits integriert sind.
- **Plattformübergreifende Entwicklung:** Mit Expo können Apps sowohl für iOS als auch für Android gleichzeitig entwickelt werden, was den Aufwand für das Testen und Verwalten der App reduziert.
- **Einfache Integration von Bibliotheken und Tools:** Expo bietet eine Reihe von vorgefertigten, gut dokumentierten Bibliotheken und APIs, die die Integration mit externen Tools erleichtern.

Nachteile von Expo:

- **Begrenzte Anpassungsmöglichkeiten:** Obwohl Expo eine einfache Handhabung bietet, gibt es Einschränkungen bei der Anpassung bestimmter nativer Module, was in einigen Fällen die Flexibilität einschränken kann.
- **Größere Dateigröße:** Apps, die mit Expo entwickelt werden, tendieren dazu, eine größere Dateigröße zu haben, was zu längeren Ladezeiten führen kann.

Diese Entscheidung hat die Grundlage für den weiteren Verlauf der Entwicklung und Implementierung der App gebildet und wurde später in der Durchführungsphase konkret umgesetzt

3.5. Designrichtlinien

Im Rahmen des App-Designs wurden spezifische Designrichtlinien entwickelt, um eine ansprechende, benutzerfreundliche und anpassungsfähige Benutzeroberfläche zu gewährleisten. Die folgenden Punkte beschreiben die wesentlichen

Designentscheidungen und -richtlinien:

3.5.1. Farbschema und Typografie

Für die App wurden zwei unterschiedliche **Farbschemata** (Dark Mode und Light Mode) entwickelt, um den Bedürfnissen der Benutzer gerecht zu werden und eine flexible Anpassung an verschiedene Benutzerpräferenzen zu ermöglichen. Diese Farbschemata passen sich an die Einstellungen des Geräts oder an benutzerspezifische Präferenzen an, sodass die Benutzeroberfläche in beiden Modi angenehm und funktional bleibt. Die



Farbgestaltung wurde auf der Grundlage einer klaren und gut lesbaren Struktur entwickelt, wobei Farben verwendet wurden, die sowohl ästhetisch ansprechend als auch funktional sind.

- **Light Mode:** Helle Hintergründe mit dunklen Textfarben sorgen für eine klare Lesbarkeit und einen hellen, angenehmen visuellen Eindruck.
- **Dark Mode:** Dunkle Hintergründe mit helleren Textfarben ermöglichen eine energiesparende Nutzung und eine geringere Belastung der Augen bei schlechten Lichtverhältnissen.

Die **Typografie** wurde ebenfalls so gewählt, dass sie sowohl im Dark Mode als auch im Light Mode gut lesbar ist. Für eine konsistente Darstellung wurden **serifenlose Schriftarten** verwendet, die modern und gut lesbar sind. Die Schriftgröße ist responsiv, um sicherzustellen, dass der Text auf unterschiedlichen Geräten und Bildschirmgrößen optimal dargestellt wird.

3.5.2. Responsive Design

Die App wurde unter Berücksichtigung von **Responsive Design**-Prinzipien entwickelt. Dies bedeutet, dass das Layout und die Benutzeroberfläche sich dynamisch an verschiedene Bildschirmgrößen und -auflösungen anpassen. Das Design berücksichtigt die Anforderungen von Geräten unterschiedlichster Größen, von Smartphones bis hin zu Tablets.

- **Flexibles Layout:** Elemente wie Buttons, Menüs und Texte passen sich der Bildschirmgröße an, sodass Benutzer auf jedem Gerät eine einheitliche Benutzererfahrung genießen können.
- **Grid-System:** Ein flexibles Grid-System sorgt dafür, dass alle UI-Komponenten korrekt ausgerichtet und proportional skaliert werden, je nachdem, ob die App auf einem kleinen Smartphone oder einem größeren Tablet angezeigt wird.

Zusätzlich wurde **Expo** als Entwicklungsframework genutzt, das von Natur aus Unterstützung für Responsive Design bietet und dafür sorgt, dass die App problemlos auf verschiedenen Geräten läuft. Mit Expo wurden die Designrichtlinien effizient umgesetzt und getestet, um eine konsistente und benutzerfreundliche Erfahrung zu gewährleisten.



4. Durchführung und Implementierung

4.1. Zielplattform und Technologie Stack

Die App wird für Android entwickelt, da die meisten Handwerkerhandys mit diesem Betriebssystem ausgestattet sind.

4.2. Durchführung und Implementierung der Datenbank

Die Datenbankstruktur wurde bereits im Abschnitt "Entwurf und Design" detailliert erläutert. Hier konzentrieren wir uns auf die technische Umsetzung:

- Die MySQL-Datenbank wurde mit MySQLWorkbench entwickelt
- Wichtige Tabellen wie Protokoll, Mitarbeiter und Wartung wurden durch Fremdschlüssel verknüpft
- Zur Performanceoptimierung wurden entsprechende Indizes für häufige Abfragen implementiert
- Diese Implementierung gewährleistet eine effiziente Speicherung und Abfrage der Wartungsinformationen.

4.3. Durchführung und Implementierung der API

Umsetzung des API-Designs

Die API wurde gemäß dem in "Entwurf und Design" beschriebenen Konzept (*siehe API-Aufbau*) implementiert. Dabei wurden folgende Aspekte realisiert:

Entwicklung mit PHP unter Verwendung des PDO-Prinzips für sichere Datenbankinteraktionen. Implementierung der geplanten Endpunkte für Wartungen, Mitarbeiter und Objektlisten. Integration der Sicherheitsaspekte wie API-Schlüssel-Authentifizierung und SSL-Verschlüsselung

Struktureller Aufbau(im Anhang unter *API-Aufbau*)

Index der API

Fungiert als Einstiegspunkt, überprüft den API-Schlüssel und leitet Anfragen an den entsprechenden Controller weiter (siehe **Fehler! Verweisquelle konnte nicht gefunden werden.** für ein Ausschnitt)



ReactController	Verarbeitet die Anfragen für verschiedene Ressourcen und enthält die Geschäftslogik(siehe <i>API ReactController</i> für ein Ausschnitt)
Gateway	Dient als Verbindungsschicht zwischen Controller und Datenbank(siehe <i>API-Gateway</i> für ein Ausschnitt)
Implementierungsdetails	Validierung und Fehlerbehandlung: Jede Anfrage durchläuft eine strikte Validierung. Ein zentrales Error-Handling-System wurde implementiert, um die Robustheit und Wartbarkeit zu erhöhen (siehe <i>API ErrorHandler</i> für ein Ausschnitt)
Logging-System	Integriert zur Protokollierung von Fehlern und erfolgreichen API-Anfragen, was die Fehlersuche und Nutzungsanalyse erleichterten
Sicherheitsmaßnahmen	➤ Verwendung von Prepared Statements gegen SQL-Injection ➤ Implementierung von Schutzmaßnahmen gegen Cross-Site-Scripting (XSS)

Hosting und Tests

Die API wurde auf dem ananas.codes-Server gehostet, um Tests in einer produktionsähnlichen Umgebung durchzuführen. Umfangreiche Tests der API-Funktionalität wurden mit Postman durchgeführt, um den gesamten Workflow von der Anfrage bis zur Antwort zu überprüfen. Diese Implementierung stellt sicher, dass die API sowohl die geplanten Funktionen erfüllt als auch die notwendigen Sicherheitsstandards einhält.

4.4. Durchführung und Implementierung der App

Die Entwicklung der App erfolgte mit Visual Studio Code unter Verwendung des Expo Frameworks, React Native und TypeScript. Die Implementierung orientierte sich an den im Entwurf festgelegten Designrichtlinien und Benutzerflüssen.

Strukturierung und Navigation



- **Ordnerstruktur:** Eine durchdachte Ordnerstruktur wurde erstellt, um eine übersichtliche Organisation der Dateien zu gewährleisten. Eine Genaue Ansicht der Ordnerstruktur befindet sich im Anhang unter *Ordnerstruktur der App*.
- **Drawer-Navigation:** Die verschiedenen Seiten der App wurden mit einer Drawer-Navigation verbunden, um ein einfaches Layout für die Navigation zu erzeugen.
- **Expo-Router:** Der Router wurde konfiguriert, um die Seitennavigation zu ermöglichen.

API-Integration

- **Verbindungstest:** Die Datei *apiService.tsx* im *service/-Ordner* wurde zur Prüfung der API-Verbindung implementiert. (*Abbildung 11 Auszug API Service in der App*).
- **Erweiterung der API-Funktionen:** Funktionen für das Posten und Löschen von Daten wurden hinzugefügt.

Hauptfunktionen

- **Konfigurationsseite (Config.tsx):**
 - Implementierung der sicheren Speicherung von Mitarbeiterkennung und API-Schlüssel in AsyncStorage.
 - Passwortschutz für sensible Daten.

Abbildung 12 ConfigScreen in der App

- **Barcode-Scan-Funktion (ScanScreen.tsx):**
 - Integration des Barcode-Scanners.
 - Speicherung der gescannten Codes in der Datenbank und lokal.
- **Protokollanzeige (Protokoll.tsx):**
 - Anzeige der gescannten Barcodes mit Zeitstempeln.
 - Implementierung von DELETE-Requests zur Entfernung von Einträgen.
 - Funktion zur manuellen Eingabe von Daten.



(

```
35 | // Verwende loadConfig im useEffect
36 | useEffect(() => {
37 |   const loadConfigurations = async () => {
38 |     try {
39 |       const { workerIdentifier, apiKey, barcodeList } = await loadConfig();
40 |       setStoredWorkerIdentifier(workerIdentifier);
41 |       setApiKey(apiKey);
42 |       setBarcodeList(barcodeList);
43 |     } catch (error) {
44 |       setModalMessage('Fehler beim Laden der Konfiguration.');
```

(

```
45 |       setErrorModalVisible(true);
46 |     }
47 |   };
48 |   loadConfigurations();
49 | }, []);
50 |
51 | const handleDelete = async (id: string) => {
52 |   try {
53 |     await deleteBarcode(apiKey, id);
54 |     const updatedList = barcodeList.filter((barcode) => barcode.id !== id);
55 |     setBarcodeList(updatedList);
56 |     await saveBarcode(updatedList);
57 |     setModalMessage('Eintrag gelöscht.');
```

(

```
58 |     setSuccessModalVisible(true);
59 |   } catch (error) {
60 |     setModalMessage('Löschen fehlgeschlagen.');
```

(

```
61 |     setErrorModalVisible(true);
62 |   }
63 | };
64 |
```

Abbildung 14 Beispiel für usestate aus der screen protollscreen)

Designumsetzung

- Implementierung des Light Mode und Dark Mode gemäß den Designrichtlinien.
- Anpassung von Farben und Schriftarten entsprechend dem Entwurf.

(Abbildung 15 Richtlinien für Responsiven Text und Farbetext)

- Umsetzung des responsiven Designs für verschiedene Bildschirmgrößen.

Weitere Anpassungen

- Erstellung der Impressum-Seite.
- Gestaltung der Startseite mit Hintergrund und Navigation zur Scan-Funktion.

Tests und Standalone-Version

- Erstellung und Test einer Standalone-Version außerhalb der Expo-Sandbox zur Überprüfung der Stabilität.



4.5. Testphase

In der Testphase wurde die App auf verschiedenen Geräten getestet von 5 unterschiedlichen Teilnehmer getestet, um die Funktionalität und Benutzerfreundlichkeit sicherzustellen. Die Ergebnisse wurden in Protokollen festgehalten, diese wurden ausgewertet und befinden sich im Anhang.

5. Ergebnisse und Fazit

5.1. Soll-Ist-Vergleich

Im Soll-Ist-Vergleich zeigt sich, dass die ursprünglichen Projektziele weitgehend erreicht wurden:

- **Zielsetzung:** Die App sollte eine digitale Lösung zur Erfassung von Wartungsarbeiten bieten.
 - **Ist-Zustand:** Die App ermöglicht nun das Scannen von QR-Codes sowie die manuelle Eingabe von Daten, was den papierbasierten Prozess ersetzt.
- **Benutzerfreundlichkeit:** Geplant war eine intuitive Benutzeroberfläche.
 - **Ist-Zustand:** Die Benutzeroberfläche wurde durch Wireframes und Mockups optimiert, was zu einer positiven Nutzererfahrung führte.
- **Sicherheitsaspekte:** Die Sicherheit der Datenübertragung war ein zentrales Anliegen.
 - **Ist-Zustand:** Implementierte Sicherheitsmaßnahmen wie API-Schlüssel und SSL-Verschlüsselung gewährleisten den Schutz sensibler Daten.

5.2. Fazit

Die Entwicklung der WiseScan App war eine anspruchsvolle, aber lehrreiche Erfahrung. Während der Implementierung traten verschiedene Herausforderungen auf, die sowohl technischer als auch organisatorischer Natur waren. Technisch gesehen war die **Datenbankstruktur** eine der ersten Hürden. Die Erstellung einer MySQL-Datenbank, die alle Wartungsinformationen speichert, erforderte sorgfältige Planung und die Verknüpfung von Tabellen durch Fremdschlüssel. Auch die **API-Entwicklung** stellte Herausforderungen dar, insbesondere bei der Implementierung von Validationen zur



Sicherstellung korrekter Eingabedaten. Ein weiteres technisches Problem war die **Integration von Sicherheitsmaßnahmen**, wie API-Schlüssel-Authentifizierung und SSL-Verschlüsselung, um unbefugten Zugriff zu verhindern. Zudem gab es Schwierigkeiten mit der **Barcode-Scan-Funktionalität**, da die ursprünglich geplante Funktionen stießen auf Probleme mit zukünftigen Expo-Versionen, was dazu führte, dass alternative Lösungen gefunden werden mussten. Aufgrund von Zeitdruck wurde eine funktionierende, aber suboptimale Lösung beibehalten. Insgesamt haben diese Herausforderungen wertvolle Lektionen für zukünftige Projekte geliefert. Die iterative Herangehensweise an die Entwicklung sowie regelmäßige Tests und Nutzerfeedback waren entscheidend für den Erfolg der WiseScan App. Die Erfahrungen aus diesem Projekt werden in zukünftigen Entwicklungen berücksichtigt, um noch effizientere und benutzerfreundlichere Lösungen zu schaffen. Was würde ich anders machen

5.3. Lessons Learned

Insgesamt haben diese Herausforderungen wertvolle Lektionen für zukünftige Projekte geliefert. Die iterative Herangehensweise an die Entwicklung sowie regelmäßige Tests und Nutzerfeedback waren entscheidend für den Erfolg der WiseScan App. Die Erfahrungen aus diesem Projekt werden in zukünftigen Entwicklungen berücksichtigt, um noch effizientere und benutzerfreundlichere Lösungen zu schaffen.

5.4. Ausblick

Für die Zukunft sind mehrere Erweiterungen geplant:

- **Erweiterung um zusätzliche Funktionen:** Weitere Features wie Push-Benachrichtigungen für Wartungserinnerungen könnten implementiert werden.
- **Optimierung der Performance:** Durch gezielte Optimierungen in der API und der Datenbankstruktur soll die Reaktionszeit weiter verbessert werden.
- **Plattformübergreifende Erweiterungen:** Eine mögliche Erweiterung auf iOS könnte in Betracht gezogen werden, um eine breitere Nutzerbasis zu erreichen.



a. Anlagen

a.1. Prozessabläufe

In diesem Abschnitt werden die spezifischen Prozesse der Anwendung detailliert beschrieben. Diese sind auf die beiden Hauptakteure – Handwerker und Administrator – zugeschnitten.

a.2. Prozesse für Handwerker

Die folgenden Prozesse beschreiben, wie Handwerker Wartungsarbeiten digital erfassen, entweder durch Scannen eines QR-Codes oder durch manuelle Dateneingabe.

➤ **Scannen und Wartungsprotokollierung**

- **Start:**
Handwerker scannt den QR-Code des Geräts mit der App.
- **Erfassung:**
Die App zeigt ein Modal (Dialogfenster) mit Gerätedetails und einem Zeitstempel an.
- **Speichern:**
Handwerker bestätigt die Angaben und speichert die Wartungsdaten.
- **Abschluss:**
Die App zeigt eine Bestätigungsmeldung an, und die Daten werden in die zentrale Datenbank hochgeladen.

➤ **Manuelle Wartungserfassung (falls Scannen nicht möglich)**

- **Start:**
Handwerker navigiert zu „Neue Wartungsarbeit hinzufügen“.
- **Erfassung:**
Handwerker gibt manuell Gerätedetails, Datum und Zeit ein.
- **Speichern und Abschluss:**
Nach Validierung der Eingaben werden die Daten in die zentrale Datenbank hochgeladen.

➤ **Löschen von Wartungsdaten**



- **Funktion:**

- Handwerker kann einzelne Wartungen nachträglich löschen.

- **Hinweis:**

- Diese Funktion wurde explizit von der Firma gewünscht.

a.3. Zusatzprozesse für Administratoren

Die Prozesse für Administratoren gewährleisten eine sichere Verwaltung der App-Konfiguration und der Benutzerzugänge.

➤ **Anmeldung in der App-Konfiguration**

- Der Administrator meldet sich mit einem Passwort an, um Zugriff auf die Konfigurationsseite zu erhalten.

➤ **Eingabe und Aktualisierung von Mitarbeiterkennungen und API-Zugangsdaten**

- Der Administrator gibt neue Mitarbeiterkennungen und API-Zugangsdaten ein oder aktualisiert bestehende, falls Änderungen erforderlich sind.

➤ **Sichern der Konfiguration**

- Nach Eingabe speichert der Administrator die Konfiguration.

Die Einstellungen sind durch ein Passwort geschützt, sodass nur berechtigte Personen Zugriff haben.

↑*Prozessabläufe*



a.4. Use-Case

(bitte einmal drehen)

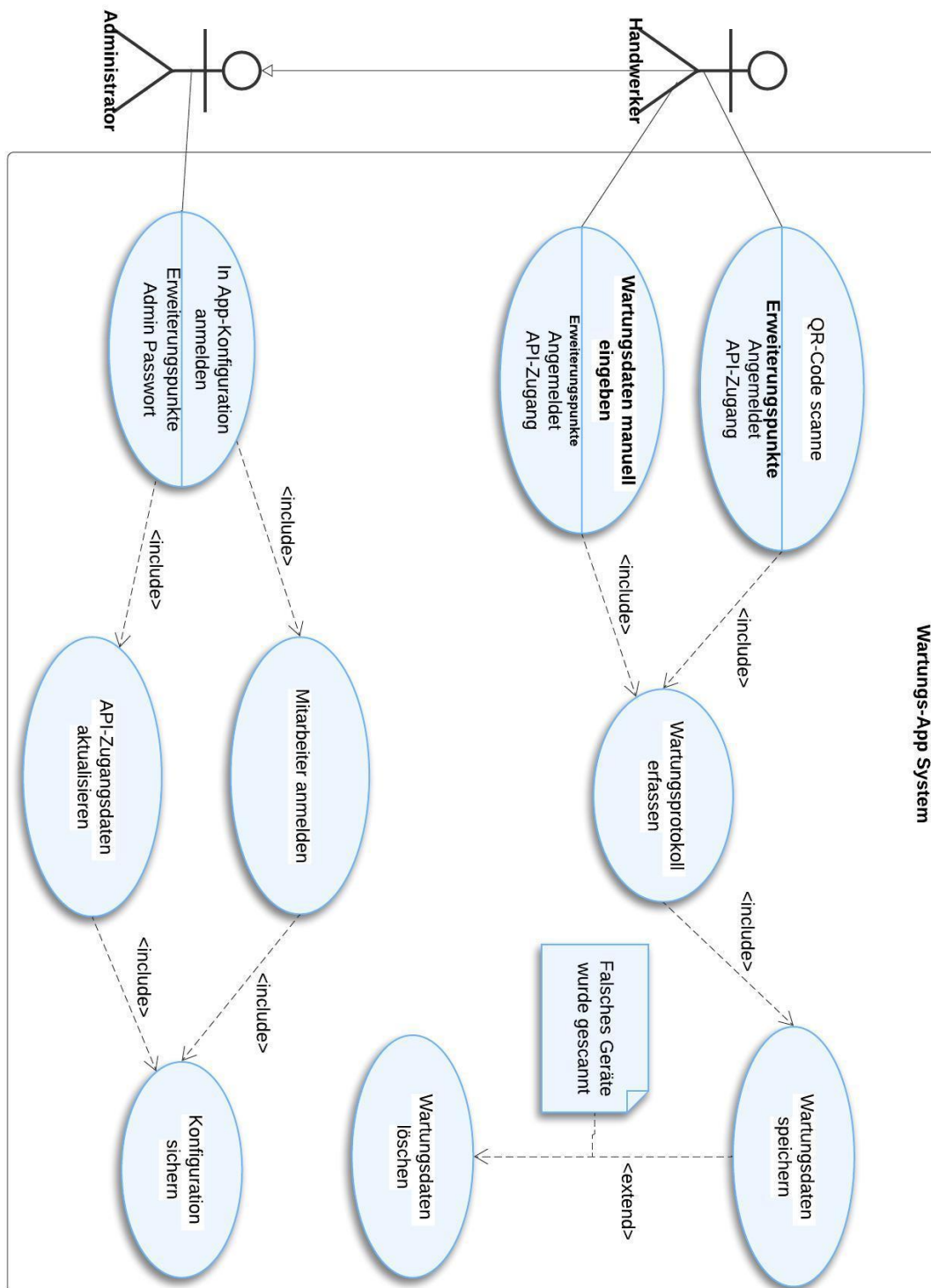


Abbildung 1 Use-Case



↑Prozessabläufe

a.5. Grobe Projektphase

Ist-Analyse und Anforderungsermittlung	8
Soll-Konzept und Design	12
Datenbankentwicklung	5
API-Entwicklung und -Implementierung	22
App-Entwicklung und Testphase	19
Dokumentation	10
Gesamt	76

↑

Zeitplanung und Projektphasen

a.6. Detaillierte Projektphase

Ist-Analyse und Anforderungsermittlung

Bezeichnung	In stunden
Analyse der vorhandenen Stakeholder-Befragungen	0,5
Zusammenfassung der Befragungsergebnisse	0,5
Erstellung eines Anforderungskatalogs	1,5
Validierung des Anforderungskatalogs mit Stakeholdern	0,5
Priorisierung der Anforderungen	1,5
Abschlussbesprechung mit den Stakeholdern	0,5
Erstellung des Lastenhefts	1
Erstellung des Pflichtenhefts	1,5
	8



Soll-Konzept und Design

Bezeichnung	In Stunden
Erstellung eines Grobkonzepts	3,5
Definieren der Hauptfunktionen und Systemarchitektur (1,5h)	
Skizzieren des Datenflussdiagramms (1h)	
Erstellen einer vorläufigen Benutzerfluss-Skizze (1h)	
Entwicklung von Mockups und Wireframes	4,5
Erstellen von Low-Fidelity Wireframes (2h)	
Entwickeln von High-Fidelity Mockups für Hauptfunktionen (2h)	
Durchführen eines internen Usability-Walkthroughs (0,5h)	
Abstimmung des Designs mit Stakeholdern	2
Präsentation des Konzepts und der Mockups (1h)	
Sammeln und Diskutieren von Feedback (1h)	
Überarbeitung und Finalisierung	2
Einarbeiten von Stakeholder-Feedback (1h)	
Finalisierung des Konzepts und der Mockups (0,5h)	
Erstellung einer Dokumentation der Design-Entscheidungen (0,5h)	
	12

Datenbankentwicklung

Bezeichnung	In Stunden
Analyse und Planung	0,5
Überprüfung der Anforderungen aus der Ist-Analyse	
Identifizierung der benötigten Datenstrukturen	
Entwurf des Datenbankschemas	1,5
Erstellung des Enhanced Entity-Relationship-Diagramm (EER)	
Definition von Tabellen, Feldern und Beziehungen	
Implementierung der Datenbankstruktur	1,5
Erstellung der Datenbanktabellen	
Definition von Constraints und Indize	
Erstellung und Einfügen von Testdaten	1
Generierung Testdatensätze	



Einfügen der Testdaten in die Datenbank	
Überprüfung und Optimierung	0,5
Durchführung von Abfragen zur Validierung der Struktur	
Optimierung von Indizes und Abfragen bei Bedarf	
	5

API-Entwicklung und -Implementierung

Bezeichnung	In Stunden
API-Design und Planung	2
Überprüfung der technischen und geschäftlichen Anforderungen	
Festlegung von API-Sprache, Benennungsstandards und Architektur	
Definition der API-Endpunkte	3
Erstellung des API-Schemas	
Festlegung von Ressourcen, Methoden und Parametern	
Implementierung der API-Funktionen	10
Codierung der API-Logik	
Integration mit Datenbanken oder anderen Diensten	
Entwicklung von Testfällen	
Implementierung automatisierter Tests	
Sicherheitskonfiguration	2
Implementierung von Authentifizierung und Autorisierung	
Einrichtung von Ratenbegrenzungen und anderen Sicherheitsmaßnahmen	
Dokumentation der API	2
Erstellung einer umfassenden API-Dokumentation	
Bereitstellung von Codebeispielen und Erklärungen	
	22



App-Entwicklung und Testphase

Bezeichnung	In Stunden
Implementierung der App-Funktionalitäten	10
Entwicklung der Benutzeroberfläche (UI) (4h)	
Implementierung der Geschäftslogik (3h)	
Integration von Drittanbieter-Bibliotheken oder -Dienstern (3h)	
Integration der API	3
Anbindung der Frontend-Komponenten an die API (1,5h)	
Sicherstellen der korrekten Datenübertragung und -verarbeitung (1,5h)	
Durchführung von Tests	4
Funktionale Tests (2h)	
Usability-Tests mit Benutzern (2h)	
Fehlerbehebung und Optimierung	2
	19

Dokumentation

Zusammenstellung der IHK-Dokumentation	10
---	----

↑

Zeitplanung und *Projektphasen*

a.7. Ressourcenplan

1) *Menschliche Ressourcen*

a) **Entwickler für Betreuung und Abnahme**

- Verantwortung: Überwachung des Projektfortschritts, Ressourcenmanagement, und Kommunikation mit Stakeholdern.
- Benötigte Zeit: 5 Stunden.

b) **Azubi (1 Person)**

- Aufgaben: Entwicklung der Datenbank, API und App sowie Integration und Tests.
- Benötigte Zeit: 30–40 Stunden/Woche.

c) **Tester**

- Aufgaben: Durchführung von Funktionstests (der App)
- Benötigte Zeit: 1 Stunden/Woche während der Testphase.

2) *Hardware*

a) **Entwicklungs-PC**

- Anforderungen: Moderne CPU, mindestens 16 GB RAM, schnelle SSD, und stabiler Internetzugang.

b) **Testgeräte**

- Android- Samsung Smartphone
- Galaxy Tab A (2016)(Android 8.10)
- Motorola edge 40 neo (Android 14)

c) **Server**

- Anforderungen: Hosting für die API und die Datenbank. Verwendet wurde ein Server bei ananas.codes für testzwecke.

3) *Software*

a) **Entwicklungsumgebungen**

- **Visual Studio Code:**
Für die App-Entwicklung mit React Native und TypeScript.



Für die API-Entwicklung.

b) Datenbankmanagement

- **MySQL Workbench:** Zum Erstellen und Verwalten der Datenbank.

c) Testing-Tools

- **Postman:** Für die Überprüfung und Simulation von API-Anfragen.
- **Expo GO:** Für App-Emulation und Debugging.

d) Versionskontrolle

- **Git** (z. B. GitHub oder GitLab): Für Versionsverwaltung und Zusammenarbeit.

e) Dokumentationstools

- MS Word/Google Docs: Für Berichte und Dokumentationen.

f) Entwurf Tool

- **Affinity Design:** Erstellen von Mockup und Wireframes

4) *Technologien und Programmiersprachen*

a) Programmiersprachen

- **TypeScript:** Für die Entwicklung der App (mit React Native).
- **PHP:** Für die Entwicklung der API.
- **SQL:** Für die Datenbankstruktur und -abfragen.

b) Frameworks und Tools

- **React Native (mit Expo Framework):** Für die plattformübergreifende Entwicklung der App.
- **Expo Router:** Für die Navigation innerhalb der App.
- **PDO (PHP Data Objects):** Für eine sichere Datenbankkommunikation in der API.
- **MySQL:** Für die Datenbankverwaltung.

5) *Sonstige Ressourcen*

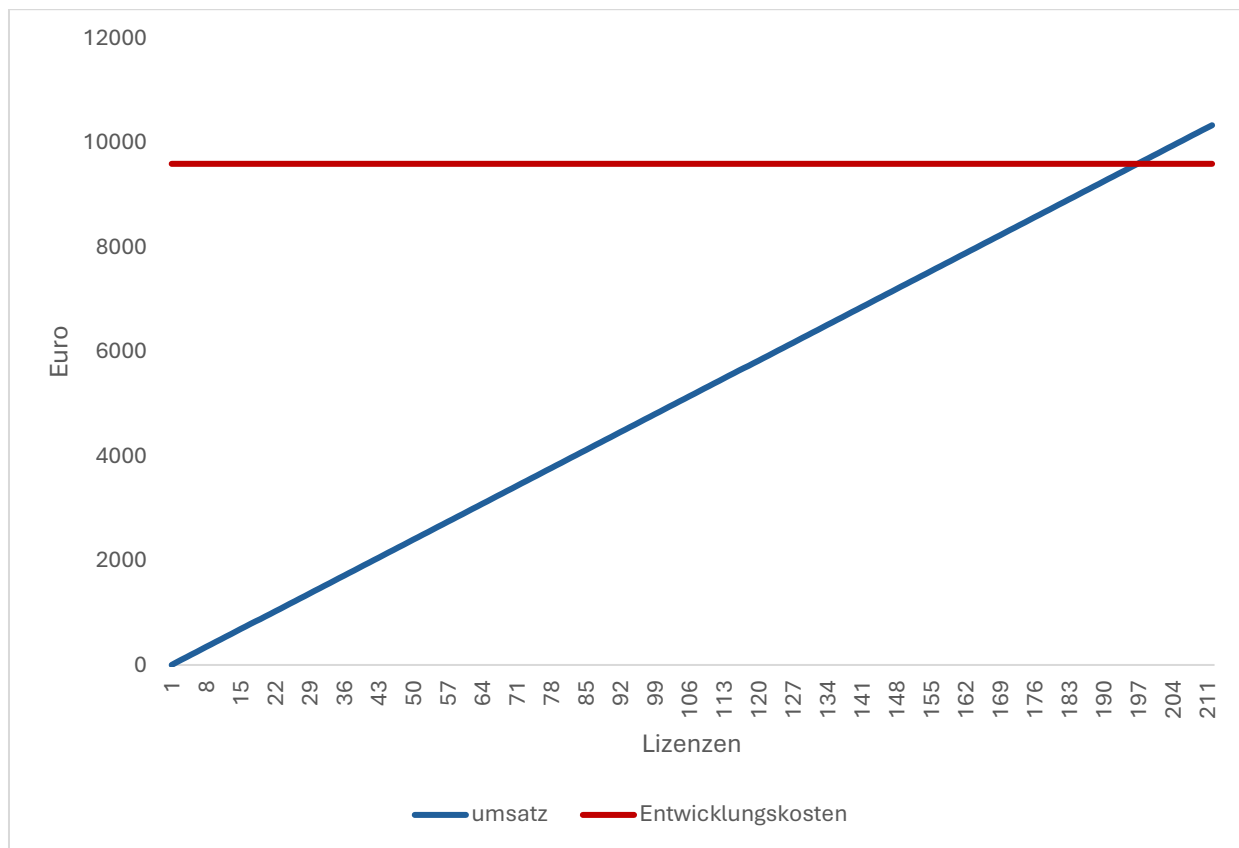
- Büro und Arbeitsplatz

6) *Zeitplan*

Siehe **Fehler! Verweisquelle konnte nicht gefunden werden.** [↑]Ressourcenplanung



a.8. Break-Event Point



↑ Break-Even-P

a.9. Kosten des Projekts

Die Kosten des Projekts setzen sich aus Personalkosten und weiteren Aufwendungen wie Büroflächen, Stromkosten und Kommunikationskosten zusammen. Um den Stundensatz von 120 € zu erreichen, werden die einzelnen Kosten wie folgt berechnet:

Personalkosten:

Die Berechnung der Personalkosten basiert auf den von der Personalabteilung festgelegten Stundensätzen:

- Mitarbeiter: 45,00 € pro Stunde
- Auszubildende: 10,50 € pro Stunde

Für die Kalkulation wird ein durchschnittlicher Stundensatz von 35,00 € zugrunde gelegt.

Ressourcenkosten:

Die im Ressourcenplan () aufgeführten Kosten werden auf pauschal 38,00 € pro Stunde über die gesamte Projektdauer festgelegt.



Sonstige Aufwendungen:

- Für Büroflächen, Stromkosten, Kommunikationskosten, Serverkosten und andere allgemeine Aufwendungen wird eine Pauschale von 1200,00 € angesetzt. Auf die Stundensätze umgelegt, ergeben sich 7 € pro Stunde ($560,00 \text{ €} \div 80 \text{ Stunden}$).

Gewinnmarge und Risikopuffer:

- Es wird eine Gewinnmarge und ein Risikopuffer in Höhe von 66,25 € pro Stunde (Differenzbetrag) hinzugefügt.

Zusammenfassung der Kosten pro Stunde:

Kostenart	Betrag (€)
Personalkosten (Durchschnitt)	35,00 €
Ressourcenkosten	38,00 €
Sonstige Aufwendungen	7,00 €
Gewinnmarge & Puffer	40 €
Gesamtkosten pro Stunde	120,00 €

Mit einer Projektdauer von 80 Stunden ergeben sich Gesamtkosten von 9.600,00 €.

↑Kosten für das Projekt



a.10. Datenfluss

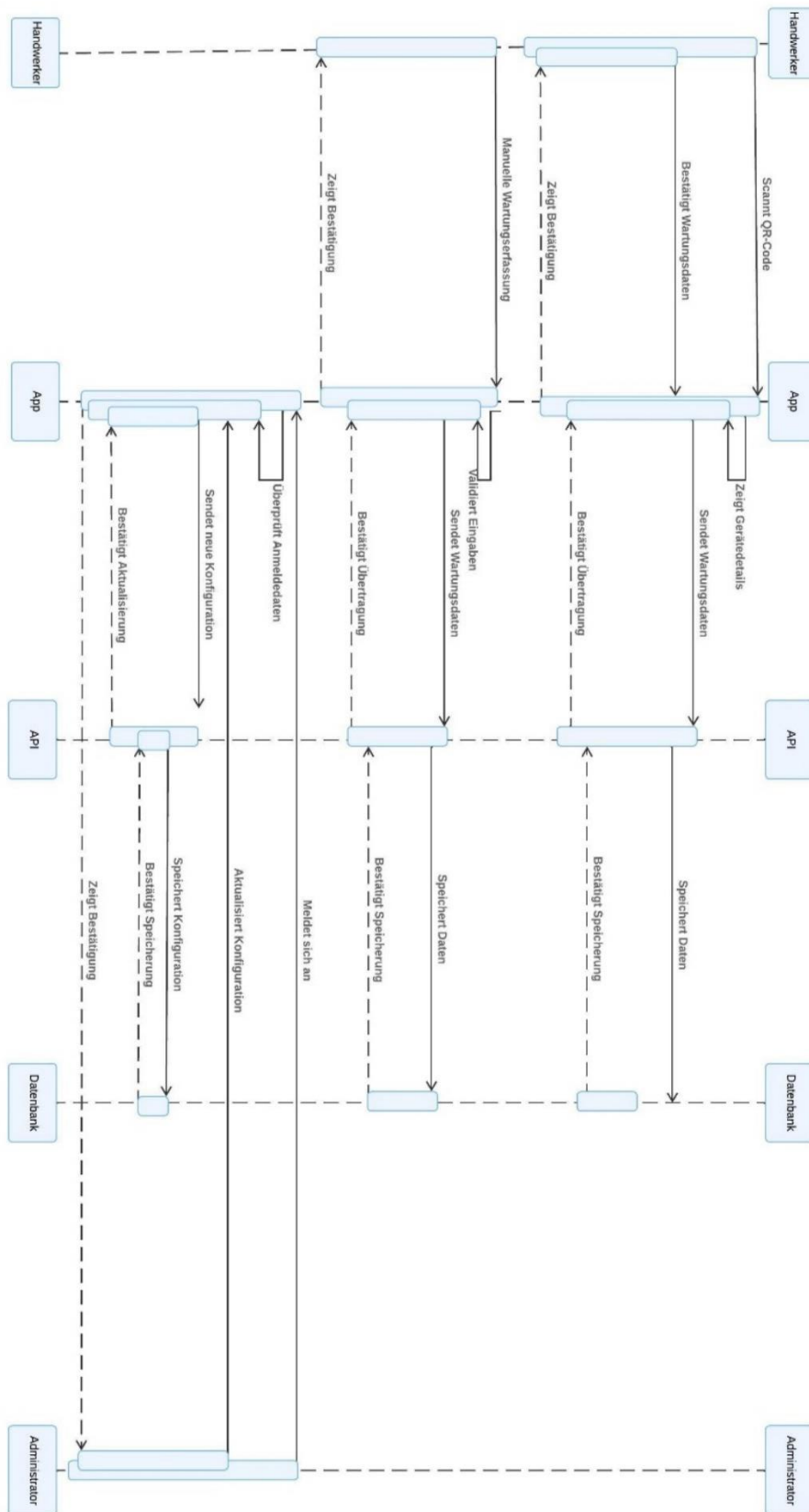


Abbildung 2 Sequenzdiagramm Datenfluss



6. ↑

Im Rahmen der Systementwicklung wurden sorgfältig durchdachte Entwurfsmethoden und Designprinzipien angewandt, um eine optimale Funktionalität und Benutzerfreundlichkeit zu gewährleisten. Der Entwurfsprozess umfasste mehrere Schlüsselschritte, die von der Systemarchitektur über das Datenbankdesign bis hin zur Benutzeroberfläche und API-Entwicklung reichen. Diese strukturierten Ansätze trugen dazu bei, ein stabiles und skalierbares System zu schaffen, das sowohl den aktuellen Anforderungen als auch zukünftigen Erweiterungen gerecht wird.

Systemarchitektur



a.11. EER-Model

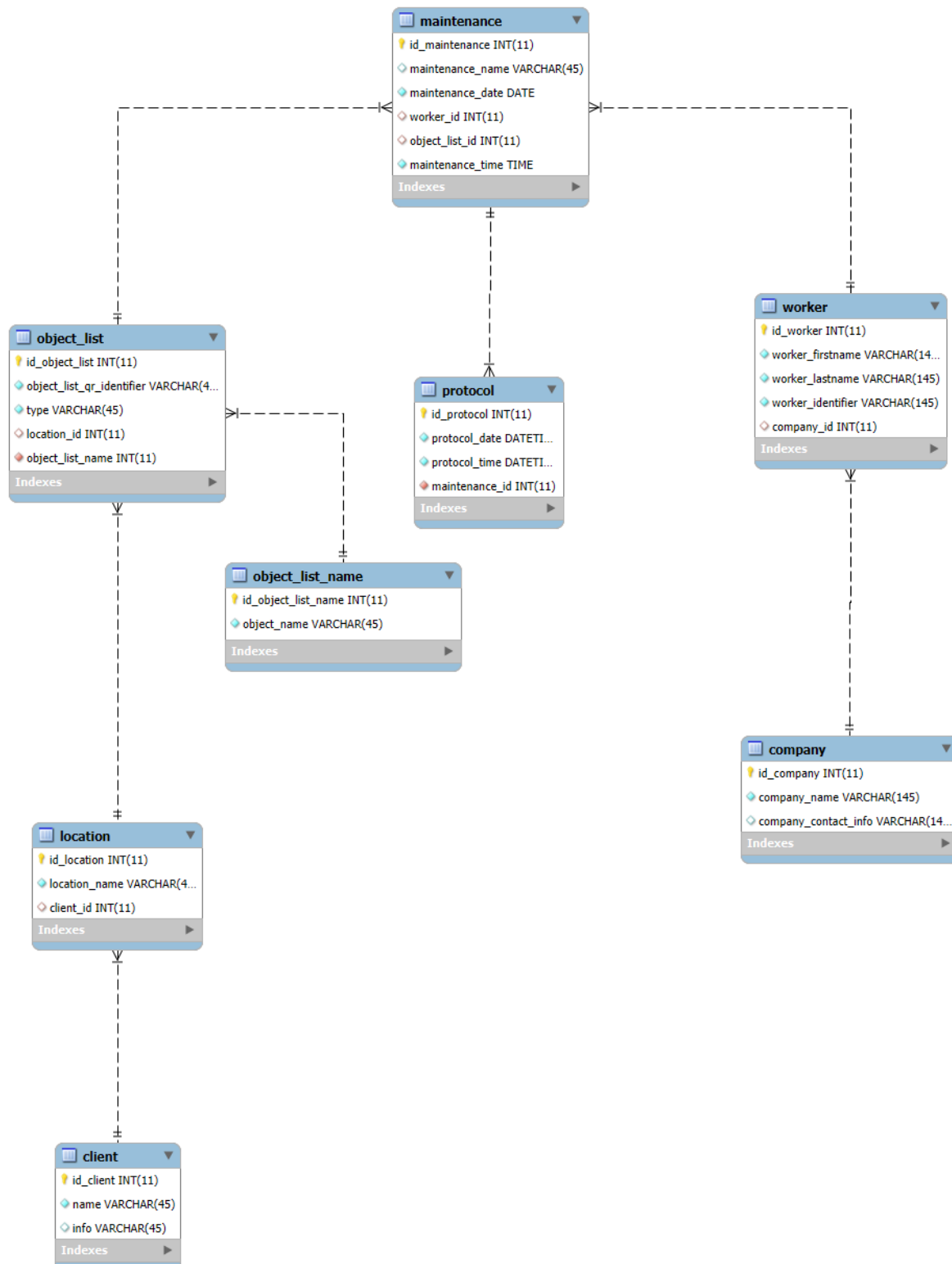


Abbildung 3 EER-Model

↑Datenbankdesign



a.12. API-Aufbau

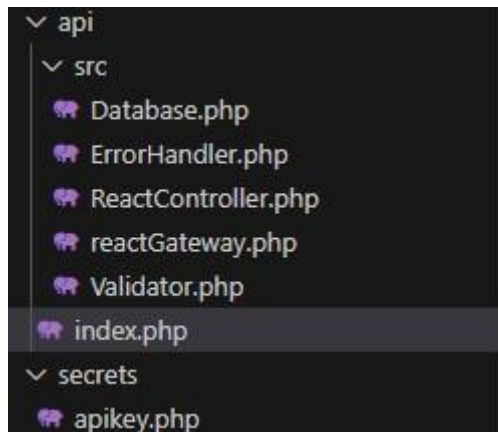


Abbildung 4 API-Ordnerstruktur

Beschreibung der Ordner und Dateien

- 1) Hauptordner
 - a) api/:
 - Enthält die gesamte API-Implementierung.
- 2) . secrets/
 - a) apikey.php:
 - Speichert den API-Schlüssel als Konstante.
- 3) . src/
 - a) Enthält alle Klassen und die Anwendungslogik.
 - b) Die Unterordner sind thematisch gegliedert, um die Trennung der Verantwortlichkeiten (Separation of Concerns) zu gewährleisten.
- 4) Controllers/
 - a) ReactController.php:
 - Enthält die API-Logik und Methoden zur Verarbeitung von HTTP-Anfragen.
 - Zuständig für die Interaktion zwischen der API und der Datenbank (über Gateways).
 - Arbeitet mit der Validator-Klasse zusammen, um Eingaben zu prüfen.
- 5) Gateways/
 - a) reactGateway.php:
 - Verbindet sich mit der Datenbank und führt Datenbankoperationen aus.
 - Beispiel:



- getAllMaintenance() für alle Wartungsarbeiten.
- createMaintenance() zum Hinzufügen neuer Wartungsarbeiten.
- Diese Schicht kapselt die Datenbanklogik und stellt Funktionen für den Controller bereit.

6) Validators/

a) Validator.php:

- Enthält Methoden, um die Daten aus den HTTP-Anfragen zu prüfen.
- Beispiel:
- Überprüfung von Barcodes.
- Validierung von IDs oder Zeitstempeln.

7) Helpers/

a) ErrorHandler.php:

- Eine zentrale Fehlerbehandlungsklasse, die API-konforme Fehlermeldungen zurückgibt.
- Beispiel:
- 404-Fehler für nicht gefundene Ressourcen.
- 400-Fehler für ungültige Eingaben.

8) 2.3.5. Database.php

a) Database.php:

- Stellt eine Verbindung zur MySQL-Datenbank her.
- Beispiel:
- Verwaltet PDO-Verbindungen.
- Gibt die Verbindung für andere Klassen frei.

9) index.php

a) index.php:

- Der Einstiegspunkt für die API.
- Prüft den API-Schlüssel, lädt die Klassen und leitet Anfragen an den ReactController weiter.
- Beispiel für den Ablauf:
- Validierung: API-Key und Ressource prüfen.



- Routing: Weiterleitung der Anfrage an den entsprechenden Controller.

a.13. Wireframe

Startseite

- App-Logo/Name oben
- Großer "Scan starten" Button in der Mitte
- Menü-Icon für die Drawer-Navigation in der oberen linken Ecke

Scan-Bildschirm

- Kameraansicht für QR-Code-Scan
- Scan-Bereich-Markierung in der Mitte
- "Manuell eingeben" Button unten

Protokoll-Ansicht

- Liste der gescannten Barcodes mit Zeitstempeln
- Lösch-Button neben jedem Eintrag
- "Neuen Eintrag hinzufügen" Button am Ende der Liste

Konfigurationsseite

- Eingabefelder für Mitarbeiter-ID und API-Schlüssel
- "Speichern" Button
- Option zum Aktivieren/Deaktivieren des Passwortschutzes

Drawer-Navigation

- Menüpunkte: Startseite, Scan, Protokoll, Einstellungen, Impressum



a.14. Mockup

a.15. Farbschema

```
1  const tintColorLight = '#03506F'; // Kräftigeres Blau
2  const tintColorDark = '#80CED7'; // Hellere Cyan
3
4  export const Colors = {
5    light: {
6
7      background: '#FAF9F6', // Sehr helles Beige statt reinem Weiß
8      tint: tintColorLight,
9
10     title: '#0E5A8A', // Mittelblau für Titel
11     subtitle: '#0096C7', // Hellere Blau für Untertitel
12     text: '#03506F', // Kräftiges Blau
13     aktzent: '#F4A261', // Warme Akzentfarbe (Orange)
14
15     listHeader: '#607D8B', // Grau für Listenüberschrift
16     listContent: '#012C59', // Dunkles Blau für Listeninhalt
17
18     icon: '#012C59',
19     tabIconDefault: '#607D8B', // Grau für Tab-Symbole
20     tabIconSelected: tintColorLight,
21   },
22   dark: {
23
24     tint: tintColorDark,
25     background: '#2e2e2e', // Dunkler Hintergrund
26
27     title: '#80CED7', // Helles Cyan für Titel
28     subtitle: '#48A9A6', // Hellere Grün-Blau für Untertitel
29     text: '#FAF9F6', // Hellere, warme Textfarbe
30     aktzent: '#F4A261', // Warme Akzentfarbe bleibt
31
32     listHeader: '#B0BEC5', // Hellgrau für Listenüberschrift
33     listContent: '#80CED7', // Hellere Akzentfarbe für Listeninhalt
34
35     icon: '#FAF9F6',
36     tabIconDefault: '#B0BEC5', // Hellgrau für Tab-Symbole
37     tabIconSelected: tintColorDark,
38   },
39 };
```

Abbildung 5 Farbschema in der App



a.16. Ablaufdiagramm app

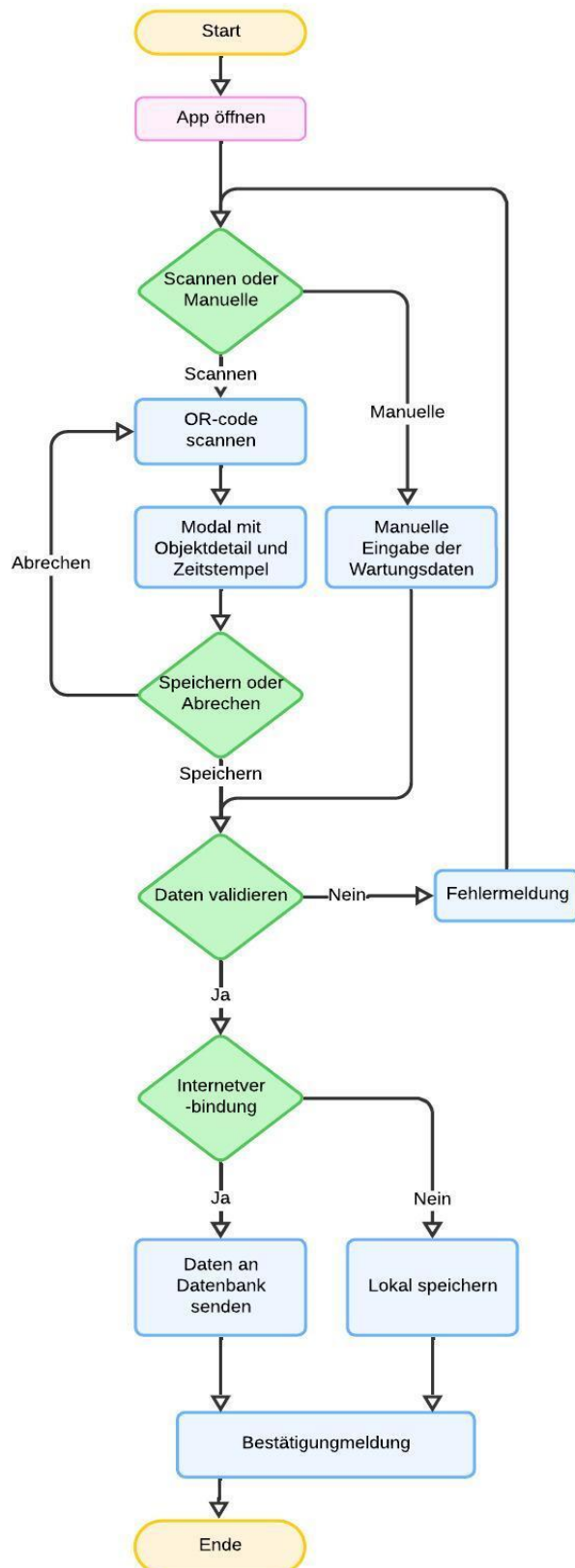


Abbildung 6 Ablaufdiagramm der App



a.17. Auszüge aus dem Code

```
api > index.php
1  <?php
2
3  declare(strict_types=1);
4
5  require_once __DIR__ . '/secrets/apikey.php';
6
7  // API-Key-Überprüfung
8  $providedApiKey = $_GET['apiKey'] ?? null;
9
10 if ($providedApiKey != API_KEY) {
11     http_response_code(403);
12     echo json_encode(["error" => "Unauthorized: Invalid API Key"]);
13     exit;
14 }
15
16 spl_autoload_register(function ($class) {
17     require __DIR__ . "/src/$class.php";
18 });
19
20 set_error_handler("ErrorHandler::handleError");
21 set_exception_handler("ErrorHandler::handleException");
22
23 header("Content-type: application/json");
24
25 // Prüfe, ob die Ressource existiert
26 $validResources = ["maintenance", "worker", "objectList"];
27
28 $resource = $_GET['resource'] ?? null;
29
30 if (!in_array($resource, $validResources)) {
31     http_response_code(404);
32     echo json_encode(["error" => "Resource not found"]);
33     exit;
34 }
```

Abbildung 7 API-Index



```

1  <?php
2  class ReactController
3  {
4
5      public function __construct(
6          private ReactGateway $gateway,
7          private Validator $validator
8      ) {}
9
10
11     //sortierung wie index sonst werden die Parameter falsch gesetzt
12     public function processRequest(
13         $method,
14         $resource,
15         ?string $workerIdentifier,
16         ?string $objectIdentifier,
17         ?string $objectIdentifierType,
18         ?string $objectID,
19         ?string $timestamp
20     ) {
21         //Es nimmt nur die parameter die weitergeben gebraucht werden in den funktionen
22         if ($resource === 'maintenance') {
23             $this->handleMaintenanceRequest($method, $workerIdentifier, $objectIdentifier, $timestamp, $objectID);
24         } elseif ($resource === 'worker') {
25             $this->handleWorkerRequestperIdentifier($method, $workerIdentifier);
26         } elseif ($resource === 'objectList') {
27             $this->handleObjectListPerLocationID($method, $locationID);
28         } else {
29             ErrorHandler::methodNotAllowed("None");
30         }
31     }
32

```

Abbildung 8 API ReactController

```

//-----POST-----
public function createMaintenance(array $data): string
{
    $object = $this->getObjectID($data["objectIdentifier"]);
    $worker = $this->getWorkerID($data["workerIdentifier"]);

    $sql = "INSERT INTO
        maintenance (
            maintenance_name,
            maintenance_date,
            worker_id,
            object_list_id,
            maintenance_time
        )
        VALUES (
            :maintenance_name,
            :maintenance_date,
            :worker_id,
            :object_list_id,
            :maintenance_time
        )";

    $stmt = $this->conn->prepare($sql);

    $stmt->bindValue(":maintenance_name", "Wartung", PDO::PARAM_STR);
    $stmt->bindValue(":maintenance_date", date('Y-m-d', strtotime($data["timestamp"])), PDO::PARAM_STR);
    $stmt->bindValue(":worker_id", $worker["id_worker"], PDO::PARAM_INT);
    $stmt->bindValue(":object_list_id", $object["id_object_list"], PDO::PARAM_INT);
    $stmt->bindValue(":maintenance_time", date('H:i:s', strtotime($data["timestamp"])), PDO::PARAM_STR);

    $stmt->execute();

    return $this->conn->lastInsertId();
}

```

Abbildung 9 API-Gateway



```

class ErrorHandler
{
    public static function handleException(Throwable $exception): void
    {
        http_response_code(500);

        echo json_encode([
            "code" => $exception->getCode(),
            "message" => $exception->getMessage(),
            "file" => $exception->getFile(),
            "line" => $exception->getLine()
        ]);
    }

    public static function handleError(
        int $erno,
        string $errstr,
        string $errfile,
        int $errline
    ): bool {
        throw new ErrorException($errstr, 0, $erno, $errfile, $errline);
    }
    /**

    public static function methodGreateScusses(string $type, string $id): void
    {
        http_response_code(201);
        header("Success: Created");
        echo json_encode(["sucess" => "$type be added to the database","id"=> $id]);
    }

```

Abbildung 10 API ErrorHandler

```

1
2 const BASE_URL = 'https://wisescan.ananas.codes/api/?apiKey=';
3
4 export const deleteBarcode = async (apiKey: string, id: string) => {
5     try {
6         const response = await fetch(`${BASE_URL}${apiKey}&resource=maintenance&objectID=${id}`, {
7             method: 'DELETE',
8             headers: {
9                 'Content-Type': 'application/json',
10                'Authorization': `Bearer ${apiKey}`,
11            },
12        });
13
14        if (!response.ok) {
15            const errorText = await response.text();
16            console.log('Fehlertext:', errorText);
17            throw new Error(`Fehler beim Löschen des Barcodes: ${errorText}`);
18        }
19
20        console.log('API-Aufruf erfolgreich');
21        return true;
22    } catch (error) {
23        console.error('Fehler beim Löschen des Barcodes:', error);
24        throw error;
25    }
26 };
27

```

Abbildung 11 Auszug API Service in der App



```

1  import React, { useEffect, useState } from 'react';
2  import ConfigForm from '@app/components/ConfigForm';
3  import LoginForm from '@app/components/LoginForm';
4  import { ThemedView } from '@app/components/ThemedView';
5  import { useAuthentication } from '@app/config/hooks/useAuthentication';
6  import { loadStoredConfig } from '@app/services/storeService';
7  import ThemedBackground from '../components/ThemedBackground';
8  import { SafeAreaView } from 'react-native-safe-area-context';
9  import { ThemedText } from '../components/ThemedText';
10 const ConfigScreen = () => {
11   const [storedWorkerIdentifier, setStoredWorkerIdentifier] = useState('');
12   const [storedApi, setStoredApi] = useState('');
13   const { password, setPassword, isAuthenticated, handleLogin } = useAuthentication('HooT');
14
15   useEffect(() => {
16     const loadConfig = async () => {
17       try {
18         const { workerIdentifier, apiKey } = await loadStoredConfig();
19         if (workerIdentifier) setStoredWorkerIdentifier(workerIdentifier);
20         if (apiKey) setStoredApi(apiKey);
21       } catch (error) {
22
23       }
24     };
25
26     loadConfig();
27   }, []);
28
29   return (
30     <ThemedBackground>
31       <SafeAreaView style={{flex:0.4}}></SafeAreaView>
32       <ThemedView style={{padding:20}}><ThemedText type="title" style={{ textAlign:'center' }}>Adminbereich</ThemedText>
33
34       {isAuthenticated ? (
35         <ConfigForm storedWorkerIdentifier={storedWorkerIdentifier} storedApi={storedApi} />
36       ) : (
37         <LoginForm password={password} setPassword={setPassword} handleLogin={handleLogin} />
38       )}
39     </ThemedView></ThemedBackground>

```

Abbildung 12 ConfigScreen in der App

```

1  import 'react-native-gesture-handler'; // Muss als erstes importiert werden
2  import DrawerLayout from './screens/_layout';
3  import { DarkTheme, DefaultTheme, ThemeProvider } from '@react-navigation/native';
4  import { useEffect } from 'react';
5  import { useFonts } from 'expo-font';
6  import 'react-native-reanimated';
7  import { useColorScheme } from './config/hooks/useColorScheme';
8  import * as SplashScreen from 'expo-splash-screen';
9
10 export default function RootLayout() {
11   const colorScheme = useColorScheme();
12   const [loaded] = useFonts({
13     SpaceMono: require('./assets/fonts/SpaceMono-Regular.ttf'),
14   });
15
16   useEffect(() => {
17     if (loaded) {
18       SplashScreen.hideAsync();
19     }
20   }, [loaded]);
21
22   if (!loaded) {
23     return null;
24   }
25
26   return (
27     <ThemeProvider value={colorScheme === 'dark' ? DarkTheme : DefaultTheme}>
28       <DrawerLayout />
29     </ThemeProvider>
30   );
31 }
32

```

Abbildung 13 Hauptlayout App



```
35 | // Verwende loadConfig im useEffect
36 | useEffect(() => {
37 |   const loadConfigurations = async () => {
38 |     try {
39 |       const { workerIdentifier, apiKey, barcodeList } = await loadConfig();
40 |       setStoredWorkerIdentifier(workerIdentifier);
41 |       setApiKey(apiKey);
42 |       setBarcodeList(barcodeList);
43 |     } catch (error) {
44 |       setModalMessage('Fehler beim Laden der Konfiguration.');
```

```
45 |       setErrorModalVisible(true);
46 |     }
47 |   };
48 |   loadConfigurations();
49 | }, []);
50 |
51 | const handleDelete = async (id: string) => {
52 |   try {
53 |     await deleteBarcode(apiKey, id);
54 |     const updatedList = barcodeList.filter((barcode) => barcode.id !== id);
55 |     setBarcodeList(updatedList);
56 |     await saveBarcode(updatedList);
57 |     setModalMessage('Eintrag gelöscht.');
```

```
58 |     setSuccessModalVisible(true);
59 |   } catch (error) {
60 |     setModalMessage('Löschen fehlgeschlagen.');
```

```
61 |     setErrorModalVisible(true);
62 |   }
63 | }
```

Abbildung 14 Beispiel für usestate aus der screen protollscreen



```

1 import { Text, type TextProps, StyleSheet } from 'react-native';
2 import { Colors } from '../config/constants/Colors'; // Import der Farben
3 import { useThemeColor } from '@app/config/hooks/useThemeColor';
4 import { RFPPercentage } from 'react-native-responsive-fontsize';
5 export type ThemedTextProps = TextProps & {
6   lightColor?: string;
7   darkColor?: string;
8   type?: 'default' | 'title' | 'defaultSemiBold' | 'subtitle' | 'link' | 'listHeader' | 'listContent';
9 };
10
11 export function ThemedText({
12   style,
13   lightColor,
14   darkColor,
15   type = 'default',
16   ...rest
17 }: ThemedTextProps) {
18   const textColor = useThemeColor({ light: Colors.light.text, dark: Colors.dark.text }, 'text');
19   const titleColor = useThemeColor({ light: Colors.light.title, dark: Colors.dark.title }, 'text');
20   const subtitleColor = useThemeColor({ light: Colors.light.subtitle, dark: Colors.dark.subtitle }, 'text');
21   const listHeaderColor = useThemeColor({ light: Colors.light.background, dark: Colors.dark.background }, 'text');
22   const listContentColor = useThemeColor({ light: Colors.light.listContent, dark: Colors.dark.listContent }, 'text');
23
24   return (
25     <Text
26       style={[
27         { color: textColor }, // Standardfarbe
28         type === 'default' && styles.default,
29         type === 'title' && [styles.title, { color: titleColor }], // Titel hat seine eigene Farbe
30         type === 'defaultSemiBold' && styles.defaultSemiBold,
31         type === 'subtitle' && [styles.subtitle, { color: subtitleColor }], // Untertitel-Farbe
32         type === 'listHeader' && [styles.listHeader, { color: listHeaderColor }], // List Header-Farbe
33         type === 'listContent' && [styles.listContent, { color: listContentColor }], // List Content-Farbe
34       ],
35       style,
36       (...rest)
37     >
38   );
39 }
40
41 const styles = StyleSheet.create({
42   default: {
43     fontSize: RFPPercentage(1.5),
44     lineHeight: RFPPercentage(3),
45   },
46   defaultSemiBold: {
47     fontSize: RFPPercentage(1.5),
48     lineHeight: RFPPercentage(3),
49     fontWeight: '600',
50     textAlign: 'center'
51   },
52   title: {
53     fontSize: RFPPercentage(3),
54     fontWeight: 'bold',
55     lineHeight: RFPPercentage(4),
56   },
57   subtitle: {
58     fontSize: RFPPercentage(2),
59     fontWeight: 'bold',
60   },
61   listHeader: {
62     fontSize: RFPPercentage(2.5),
63     lineHeight: RFPPercentage(3.5),
64   },
65   listContent: {
66     fontSize: RFPPercentage(1.5),
67     lineHeight: RFPPercentage(2),
68   },
69 });
70

```

Abbildung 15 Richtlinien für Responsiven Text und Farbetext



a.18. Ordnerstruktur der App

➤ **assets/**

Beinhaltet statische Dateien wie:

- fonts/: Schriftarten, die in der App verwendet werden.
- images/: Bilder und andere grafische Ressourcen.

➤ **components/**

Enthält wiederverwendbare UI-

Komponenten, z. B.:

- Themed Components: Automatisch anpassbar an das Farbschema der App.
- ThemedView.tsx, ThemedText.tsx, ThemedButton.tsx, ThemedBackground.tsx

➤ **Weitere Komponenten:**

- BarcodeListItem.tsx: Darstellung eines einzelnen Listenelements (z. B. Barcode).
- CustomModal.tsx, DeleteModal.tsx, ScanModal.tsx: Unterschiedliche modale Dialoge.

➤ **screens/**

Beinhaltet die Hauptbildschirme der App:

- _layout.tsx: Gemeinsames Layout für Bildschirme (z. B. Navigation).
- configScreen.tsx: Bildschirm für Einstellungen und Konfigurationen.

```
▼ app
  ▼ assets
    > fonts
    > images
  ▼ components
    ▼ modal
      CustomModal.tsx
      DeleteModal.tsx
      ManuelModal.tsx
      ScanModal.tsx
    BarcodeListItem.tsx
    ConfigForm.tsx
    LoginForm.tsx
    ThemedBackground.tsx
    ThemedButtons.tsx
    ThemedFlatList.tsx
    ThemedText.tsx
    ThemedTextInput.tsx
    ThemedView.tsx
  ▼ config
    ▼ constants
      TS Colors.ts
    ▼ hooks
      useAuthentication.tsx
      TS useColorScheme.ts
      TS useThemeColor.ts
  ▼ screens
    _layout.tsx
    configScreen.tsx
    impressumScreen.tsx
    index.tsx
    protollScreen.tsx
    scanScreen.tsx
  ▼ services
    apiService.tsx
    storeService.tsx
    _layout.tsx
```



- `impressumScreen.tsx`: Zeigt rechtliche Informationen (Impressum, Datenschutz).
- `index.tsx`: Hauptbildschirm oder Einstiegspunkt der App.
- `protocolScreen.tsx`: Verwaltung und Anzeige von Protokollen.
- `scanScreen.tsx`: Funktionen für das Scannen von Barcodes oder QR-Codes.

➤ **services/**

Enthält Geschäftslogik und API-Verwaltung:

- `apiService.tsx`: Führt API-Aufrufe aus und verarbeitet Antworten.
- `storeService.tsx`: Verwaltung lokaler Daten (z. B. mit `AsyncStorage`).
- `_layout.tsx`: Möglicherweise für gemeinsam genutzte Layoutlogik in Services.

➤ **hooks/**

Enthält benutzerdefinierte Hooks:

- `useAuthentication.tsx`: Verwaltung von Authentifizierungslogik.
- `useColorScheme.ts`: Erkennung des aktuellen Farbschemas (z. B. hell/dunkel).
- `useThemeColor.ts`: Liefert Farben entsprechend dem aktuellen Theme.

➤ **constants/**

Beinhaltet unveränderliche Werte:

- `Colors.ts`: Verwaltung von Farbwerten für die App (z. B. für helles und dunkles Theme).

.Wichtige App-Bestandteile

➤ **Themed Components**

Passen das Aussehen automatisch an das Farbschema der App an. Beispiele:

- `ThemedView`, `ThemedText`, `ThemedButton`.

➤ **Wiederverwendbare Komponenten**

Häufig genutzte UI-Elemente:

- `BarcodeListItem`: Komponente zur Darstellung eines Listenelements (z. B. Barcodes).
- `CustomModal`: Flexibles und anpassbares Modal.

➤ **Hooks**

Benutzerdefinierte Hooks:

- `useThemeColor`: Anpassung von Farben basierend auf dem Thema.
- `useAuthentication`: Verwaltung des Login-Status.



React Native Hooks:

- useState: Lokale Zustandsverwaltung.
- useEffect: Ausführung von Effekten bei Änderungen.

➤ .Services**Zentral für die API- und Datenlogik:**

- apiService: Führt API-Kommunikation durch (z. B. Daten vom Server abrufen).
- storeService: Verwaltung und Speicherung von Daten lokal auf dem Gerät (z. B. mit AsyncStorage).

➤ State Management**Lokale Zustandsverwaltung mit:**

- useState: Für einfache Zustände.
- useEffect: Für Nebenwirkungen, z. B. API-Aufrufe.

➤ Theme und Style Management

- Colors.ts: Definiert Farbpaletten für helles und dunkles Theme.
- Styles zentral verwaltet, um ein konsistentes Design sicherzustellen.

➤ Bildschirme (Screens)

- ProtocolScreen: Darstellung und Verwaltung von Protokollen.
- IndexScreen: Übersicht oder Startbildschirm.
- ScanScreen: Scannen von Barcodes/QR-Codes.
- ConfigScreen: Einstellungen und Konfiguration.
- ImpressumScreen: Rechtliche Informationen.

